

Assignment 05

For this assignment you will be writing a series of programs which should be saved independently as their own ".py" file, except for **OPTIONAL** Problem 4, which can be a .doc or .pdf file. The filename you should use for each program should have the same naming convention as previous assignments. When you're finished you should submit your programs to the Assignment #5 category inside of Brightspace.

Problem 0: Pizza Party

You are arranging a pizza party for all of your friends. You want to figure out how much pizza you can order without exceeding your budget. Write a program that accepts the number of people attending your party, the budget for your party, the price per slice of pizza, and the price per pie of pizza. Your program should then identify how many slices each person wants and determine how many individual slices and pies to order without exceeding your budget. If you exceed your budget, the program should warn the user they can't place their order. Note that for the purpose of this program a pizza pie contains 8 slices. And note that the restaurant you are ordering from will not sell you more than 7 individual slices at a time (if you need to purchase 8 slices you will have to buy a whole pie).

You do not need to validate the first four input statements (total budget, cost per slice, cost per pie and # of people attending party). You do have to validate the user's input when prompting them for the number of slices for each person coming to the party (i.e. Enter number of slices for person #1).

Example input/output

```
Enter budget for your party: 100
Cost per slice of pizza: 2.50
Cost per whole pizza pie (8 slices): 12.50
How many people will be attending your party? 10
Enter number of slices for person #1: -2
Not a valid entry, try again!

Enter number of slices for person #1: 2
Enter number of slices for person #2: 3
Enter number of slices for person #3: 4
Enter number of slices for person #4: 2
Enter number of slices for person #5: 3
Enter number of slices for person #6: 4
Enter number of slices for person #7: 2
```

Enter number of slices for person #8: 1
Enter number of slices for person #9: 5
Enter number of slices for person #10: 4
You should purchase 3 pies and 6 slices
Your total cost will be: 52.50
You will still have 47.50 left after your order

Enter budget for your party: 10
Cost per slice of pizza: 2.50
Cost per whole pizza pie (8 slices): 12.50
How many people will be attending your party? 4
Enter number of slices for person #1: 1
Enter number of slices for person #2: 1
Enter number of slices for person #3: 1
Enter number of slices for person #4: 1
You should purchase 0 pies and 4 slices
Your total cost will be: 10.00
You will have no money left after your order.

Enter budget for your party: 10
Cost per slice of pizza: 2.50
Cost per whole pizza pie (8 slices): 12.50
How many people will be attending your party? 4
Enter number of slices for person #1: 2
Enter number of slices for person #2: 2
Enter number of slices for person #3: 2
Enter number of slices for person #4: 2
Your order cannot be completed.
You would need to purchase 1 pies and 0 slices

This would put you over budget by 2.50

Problem 0 Points (22)

- Only accepts positive numbers for slices using a loop to validate (3 points)
- Uses a loop to get number of slices for each person (5 points)
- Uses input function to get budget, cost per slice of pizza, cost per pie, number of people, and number of slices per person (5 points)
- Correctly calculates number of pies and slices – if more than 8 slices are purchased, gets a pie, and the remainder after pies are slices (3 points)
- Calculates total cost and leftover money, with 3 possible outcomes: (1) leftover money, (2) exactly the budget was used, (3) went over budget (6 points)

Problem 1: Step by Step

Write a program that produces the following pattern **using nested loops**:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 . . . n
```

The program will ask the user for an upper limit. Your program must ensure that the number is positive.

The program must start at 1, print “1” on a line, and for each number up to and including the number the user specifies, will print out an additional number as well as all of the number preceding it on the following line.

Here's a sample running of your program:

```
Pick a number to go up to: -5
Invalid, try again.
Pick a number to go up to: 10
1
1 2
```

```
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9 10
```

Problem 1 Points (18)

- Only accepts a positive number for user input, uses loop to validate this (3 points)
- Pattern starts at 1 (2 points)
- Pattern ends at the number the user picks, INCLUDING that number in the final line (4 points)
- Each line in the pattern has one more number than the line before it (4 points)
- Uses a nested loop to complete this problem (5 points)

Problem 2: Prime Numbers

Note: Each part of this problem should be submitted as a separate .py file!

Problem 2a: Prime Number Finder

Write a program that prompts the user to enter in a positive number.

Next, determine if the given number is a prime number. A prime number is a number that has no positive divisors other than 1 and itself. For example, 5 is prime because the only numbers that evenly divide into 5 are 1 and 5. 6, however, is not prime because 1, 2, 3 and 6 are all divisors of 6.

Here's a sample running of the program:

```
Enter a positive number to test: 5

2 is NOT a divisor of 5 ... continuing
3 is NOT a divisor of 5 ... continuing
4 is NOT a divisor of 5 ... continuing

5 is a prime number!
```

And here's another running:

Enter a positive number to test: 9

2 is NOT a divisor of 9 ... continuing

3 is a divisor of 9 ... stopping

9 is not a prime number.

Some notes on your program:

- 1 is technically not a prime number.
- Once you find a number that evenly divides into your test number you do not need to continue testing additional numbers - the number cannot be prime.

Problem 2b: Find all Prime Numbers between 1 and 1000

Next, make a copy of Part A and update it so that the program now finds all prime numbers between 1 and 1000. Here's a sample running of your program:

1 is technically not a prime number.

2 is a prime number!

3 is a prime number!

5 is a prime number!

7 is a prime number!

11 is a prime number!

... cut ...

977 is a prime number!

983 is a prime number!

991 is a prime number!

997 is a prime number!

Problem 2c: Custom Number Range

Make a copy of Part B and update it so that the user can choose to examine a specific range of numbers for prime numbers. Here's a sample running of your program:

Start number: 5

End number: -5

Start and end must be positive

Start number: 5

End number: 3

End number must be greater than start number

Start number: 5

End number: 23

5

7

11

13

17

19

23

Some notes on your program:

- You need to ensure that the start and end numbers are both positive.
- You also need to ensure that the start number is less than the end number.

Problem 2 Points (30)

- Part 2a: Only accepts a positive number for user input, uses loop to validate this (3 point)
- Part 2a: For prime numbers, prints all numbers before that number and says they are not divisors of that number (4 points)
- Part 2a: For non-primes, prints all numbers up to the first divisor of that number and then stops and says the number is not a prime number (4 points)
- Part 2b: Prints prime numbers between 1 and 1000
 - Prints "1 is technically not a prime number" (2 points)
 - 2 is included as a prime number (2 points)
 - Prints all other prime numbers (5 points)
- Part 2c: Uses loop to validate that the second number is larger than the first (3 points)
- Part 2c: Passes a test case where start is > 1 and end is < 1000; prints all primes including and between those numbers (6 points)

Problem 3: Multiplication Table

Write a program that prompts the user for two integers that are greater than or equal to zero. Ensure that the first integer is less than the second integer. Next, write a program that generates a "multiplication table" using these numbers that computes the product of all possible values - use the output below as a guide:

Lowest number: -5

Lowest number must be 0 or greater

Lowest number: 0

Highest number: 0

Highest number must be larger than lowest number!

Highest number: 5

```
+  0  1  2  3  4  5
-----
0 | 0  0  0  0  0  0
1 | 0  1  2  3  4  5
2 | 0  2  4  6  8 10
3 | 0  3  6  9 12 15
4 | 0  4  8 12 16 20
5 | 0  5 10 15 20 25
```

Here are some hints to get you started:

- Don't worry about formatting your table until you have a basic version printing out correctly.
- Start by generating the first line of output (the '+' sign and the column headings)
- Next, generate the left most column (the row headings)
- Big hint: Python prints from top to bottom, left to right. Before moving onto the next row, what do you have to do? Do you need another loop here?
- When you're ready to format your table you need to ensure that no matter what there will always be room for all of your numbers. Hint: you might need to count the size of your largest possible value and use that as part of the "format" function. For example, notice how the spacing in the following tables always works out no matter the size of the values being printed:

Lowest number: 123

Highest number: 128

```
+  123  124  125  126  127  128
```

```

-----
123 | 15129 15252 15375 15498 15621 15744
124 | 15252 15376 15500 15624 15748 15872
125 | 15375 15500 15625 15750 15875 16000
126 | 15498 15624 15750 15876 16002 16128
127 | 15621 15748 15875 16002 16129 16256
128 | 15744 15872 16000 16128 16256 16384

```

Problem 3 Points (30)

- Loop only allows numbers of 0 or larger (3 points)
- Loop ensures highest number is bigger than lowest number (3 points)
- Nested loop is used to print rows and columns (5 points)
- Dashes and lines are used to separate the row and column headers from the results (4 points)
- Each output number is the result of multiplying the number in the row by the number in the column (6 points)
- Numbers are correctly formatted to be right-aligned (6 points)
- Dashes on header are adjusted to extend to the end of the row regardless of length of input (3 points)

Problem 4: Optional Bonus – Testing, Testing...

As you write more complicated programs, it will become more and more important to test your code. If you continue with CS classes, you will learn how to do this more formally. For now, we can test our code by **coming up with different inputs that we expect to trigger different outputs**. We want to make sure that we test for every possible outcome of our program. **This problem is worth up to 15 bonus points on this assignment.**

For example, in Assignment #3, we had the following question:

Daily Schedule

You will be writing a program that first asks the user for a time of day. For simplicity, the user's schedule will be in military time. You can assume that the user will **enter an integer (i.e. 0 = 12AM, 7 = 7AM, 13 = 1PM, 22 = 11PM)** for this input. **However, you cannot assume this number will be between 0 and 23 and will need to make sure the time is valid (see sample run below).** Next, you will ask **them for the day of week**. Depending on whether it is a weekday (Monday-Friday) or a weekend (Saturday or Sunday), there are some differences to the schedule. You can assume the user will always enter a valid day of the week (i.e. Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday) for this input.

Here is the daily schedule you will use to tell the user what activity they should be doing at a given day/time:

- 8AM-3PM (8-15): Class
- 4PM-6PM (16-18): Free time
- 7PM-9PM (19-21):
 - Weekday: Cook and eat dinner
 - Weekend: Go out for dinner with friends
- 10PM-7AM (22-23, 0-7): Sleep

This question provided the scaffolding for us – there were 6 possible outcomes that the program could have: (1) invalid time, (2) class time, (3) free time, (4) cook and eat dinner, (5) go out for dinner with friends, and (6) sleep.

In order to grade your assignments, we used the following test cases with the following expected outcomes:

Input (Time; Day)	Expected Output
-3; Wednesday	You have entered an invalid time!
2; Thursday	During this time, you sleep
13; Monday	During this time, you have class
16; Sunday	During this time, you have free time
20; Friday	During this time, you cook and eat dinner
21; Saturday	During this time, you go out for dinner with friends

These were just a few possibilities and not exhaustive. For a more complete list, we could have used something like this (again, just examples, but **this time takes all possibilities into account**):

Input (Time; Day)	Expected Output	Why?
-3; Wednesday	You have entered an invalid time!	Time < 0, regardless of day of week
26; Saturday	You have entered an invalid time!	Time > 23, regardless of day of week
2; Thursday	During this time, you sleep	Time >= 0 and time <= 7, regardless of day of week
22; Saturday	During this time, you sleep	Time >= 22 and time <= 23, regardless of day of week
13; Monday	During this time, you have class	Time >= 8 and time <= 15, regardless of day of week
9; Saturday	During this time, you have class	Time >= 8 and time <= 15, regardless of day of week
16; Sunday	During this time, you have free time	Time >= 16 and time <= 18, regardless of day of week
18; Tuesday	During this time, you have free time	Time >= 16 and time <= 18, regardless of day of week
20; Friday	During this time, you cook and eat dinner	Time >= 19 and time <= 21, week day

21; Saturday	During this time, you go out for dinner with friends	Time ≥ 19 and time ≤ 21 , weekend
--------------	--	---

Here, we don't only think about the possible outcomes but also the different ways we may need to test that outcome. We need to consider time and week night vs. week day as well as special cases (ex. Sleep needs to work when time is ≥ 0 and ≤ 7 AND when time is ≥ 22 and ≤ 23).

For this problem, you will be given 2 different programs and asked to complete similar tables as the descriptive (3 column) table above. Please save these tables as a .doc or .pdf for your submission.

- a. **Special Numbers – You should test 7 different inputs for this program and assume the user will always enter a number**

```
#set valid to False and use to control while loop
valid = False
while valid == False:
    num = int(input("Tell me your number:"))
    #needs a positive number to proceed
    if num >= 0:
        valid = True
        #otherwise, ask for another number
    else:
        print("Invalid, try again!")
#even numbers
if num%2 == 0:
    print("Your number is even!")
    #divisible by 4
    if num%4 == 0:
        print("Cool!")
        #divisible by 5
        if num%5 == 0:
            print("Wow!")
#odd numbers
else:
    print("Your number is odd!")
```

```
#divisible by 5
if num%5 == 0:
    print("Great!")
```

b. Two Dice – You should test different 8 inputs for this program and assume the user will always enter numbers. (one input counts as two numbers – one for each dice)

```
#ask the user what he rolled on his first dice
valid1 = False
while valid1 == False:
    dice1 = int(input("What was your first roll?"))
    if dice1 >= 1 and dice1 <= 6:
        valid1 = True
    else:
        print("Invalid, try again!")
#ask the user what he rolled on his second dice
valid2 = False
while valid2 == False:
    dice2 = int(input("What was your second roll?"))
    if dice2 >= 1 and dice2 <= 6:
        valid2 = True
    else:
        print("Invalid, try again!")
#two even rolls
if dice1%2 == 0 and dice2%2 == 0:
    print("Evens!")
#two odd rolls
if dice1%2 == 1 and dice2%2 == 1:
    print("Odds!")
#sum roll
if dice1 + dice2 == 6:
    print("Sum!")
#high/low
if dice1 == 1 and dice2 == 6 or dice1 == 6 and dice2 == 1:
```

```
print("High/Low roll!")  
  
#snake eyes  
  
if dice1 == 1 and dice2 == 1:  
    print("Snake Eyes!")
```

- For this problem, you can earn up to 15 points (1 point per test case)